

Séance JavaScript — Projet de groupe (experts → mutualisation)

Objectif : comprendre, expliquer et réutiliser des scripts pour rendre une page web interactive.

Matériel / supports

Liens Capytale (progressifs) + script (A→F)

- Page HTML support (puis une page HTML « à réparer »)
- Document d'aide JavaScript (distribué après un temps de recherche)

Deux temps, un même fil rouge

Différenciation par scripts + apprentissage par l'explication (jigsaw).

Temps 1 : Groupes experts

- Un script par groupe (niveau adapté)
- Comprendre + commenter le code
- Identifier les éléments HTML attendus
- Préparer une explication courte

Temps 2 : Groupes projets

- Groupes hétérogènes (1 expert par script)
- Restitution orale entre pairs
- Intégration : une page = plusieurs interactions
- Tests + corrections + bilan

Lancement

0 - 10 min

Consignes + groupes

Experts

10 - 55 min

Analyse + test

Aide

≈ 35 min

Document d'appui

Projets

55 - 110 min

Mutualiser + intégrer

Bilan

110 - 120 min

Synthèse

Constitution des groupes & distribution initiale

Objectif : mettre chaque élève en situation de réussite, sans stigmatisation.

Déroulement (10 min)

- Annonce : “Vous devenez expert d’un script, puis vous l’enseignerez à d’autres.”
- Groupes experts (3-4) : plutôt homogènes en niveau
- Attribution d’un script (A→ F) + 1er lien Capytale (J S seul + consignes)
- Rappel de la règle : on ne modifie pas le J S (on comprend / on commente)

Analyse autonome du script (sans document d'aide)

Objectif : entrer dans le code, repérer l'essentiel, formuler des hypothèses.

Consignes élèves

- Lire le script en entier
- Repérer : variables, sélecteurs (id/class), événements
- Expliquer : "Quand l'utilisateur fait X, la page fait Y"
- Ajouter des commentaires simples (en français)

Rôle enseignant

- Circuler et questionner (sans donner la solution)
- Vérifier que chaque groupe d'élèves comprend le "but"
- Débloquer par indices : où le script cherche-t-il le HTML ?
- Repérer les besoins pour l'aide (étape suivante)

Distribution du document d'aide (après un temps de recherche)

Objectif : sécuriser la compréhension sans court-circuiter la démarche.

Ce que l'aide doit permettre

- Comprendre : document / DOM / sélection (`getElementById`, `querySelector`)
- Comprendre : événement (`click`, `input`) → fonction exécutée
- Comprendre : modification (`style.display` / `classList`)
- Revenir au script et améliorer les commentaires

Capitale 2 : même script + document d'aide

Tester : HTML + JavaScript

Objectif : voir le script “en action” et relier le code aux éléments HTML.

Consignes (groupes experts)

- Ouvrir le lien Capytale 3 (HTML + JavaScript)
- Identifier exactement : le bouton / champ / zone modifiée
- Relever les attributs indispensables (id, class, data-*)
- Vérifier : l'interaction attendue correspond bien à votre explication

Retour en groupes hétérogènes : restitution et intégration

Objectif : apprendre en expliquant, puis construire une page commune.

Restitution (20-25 min)

- Chaque expert explique son script
- Les autres posent des questions
- On liste les éléments HTML nécessaires
- On repère les conflits possibles (id doublons)

Intégration (15-20 min)

- Les scripts ensemble (mêmes fichiers)
- Ajouter / ajuster le HTML support
- Tester les interactions

Défi Capytale 4 : réactiver les interactions (HTML à compléter)

Objectif : transférer la compréhension sur une autre mise en page.

Règles du défi

- Le JavaScript est inchangé : interdiction de le modifier
- Vous corrigez uniquement le HTML (id, class, data-target, etc.)
- Objectif : retrouver les 6 interactions actives
- Validation : chaque script fonctionne (tests rapides)

Critères de réussite & erreurs fréquentes

Pour guider le travail et la correction rapide.

Réussite

- J e peux expliquer mon script en 30 secondes
- J e sais quels éléments HTML sont indispensables
- J e sais ce qui est modifié (classe / style / contenu)
- La page commune fonctionne (tests)

Erreurs fréquentes

- id/class ne correspondent pas au J S
- Scripts chargés avant le HTML (si `<script>` est trop haut)
- Conflits : deux éléments avec le même id
- On “bricole” sans commenter / sans comprendre

Trace écrite (3 phrases)

À copier / coller dans le cahier ou sur l'ENT.

- 1) HTML décrit la structure de la page, CSS définit son apparence.
- 2) JavaScript rend la page interactive en réagissant à des événements (clic, saisie...).
- 3) Un script fonctionne seulement si le HTML contient les bons éléments (id, class, data-*).

Sortie (2 min) : "Quelle est LA chose que tu sais faire maintenant que tu ne savais pas faire avant ?"